

Topic: Recursive Queries

Q. What are Recursive Queries in SQL? Explain the concept of Common Table Expressions (CTE) and how a Recursive CTE works with an example.

📌 Definition to Remember

A **Recursive Query** is a query that **repeatedly executes itself** until a termination condition is met, used to traverse hierarchical or tree-structured data. It is implemented in SQL using a **Recursive CTE** (Common Table Expression) with the **WITH RECURSIVE** keyword, consisting of an **Anchor Member** and a **Recursive Member** joined by **UNION ALL**.

Recursive Queries & Common Table Expressions (CTE)

A **CTE** is a temporary, named result set defined using the **WITH** keyword. It exists only during the execution of a single SQL statement.

* Used to simplify complex queries by breaking them into readable named blocks.

* **Syntax:** **WITH CTE_Name AS (SELECT ...)**

2. Structure of a Recursive CTE

A recursive CTE has exactly **3 components**:

```
WITH RECURSIVE CTE_Name AS (  
    | ANCHOR MEMBER | ← Base case (executes once, gives R0)  
    | SELECT ...    |  
    | UNION ALL     | ← Connects anchor + recursive  
    | RECURSIVE MEMBER | ← References CTE itself (gives R1, R2, ...)  
    | SELECT ... JOIN CTE_Name ... |  
    | ) |  
SELECT * FROM CTE_Name;
```

3. Working of a Recursive Query

1. **Anchor Member** executes first → produces base result set R₀.
2. **Recursive Member** uses R₀ as input → produces R₁.
3. Recursive Member uses R₁ → produces R₂.
4. Repeats until the Recursive Member returns an **empty set** (termination).
5. Final result = **UNION ALL** of all sets: R₀ + R₁ + R₂ + ...

4. Example — Organizational Hierarchy

Table: **Employee (EmpID, Name, ManagerID)** — Find all levels of management.

```

WITH RECURSIVE OrgChart AS (
  -- Anchor: Start with the CEO (no manager)
  SELECT EmpID, Name, ManagerID, 1 AS Level
  FROM Employee
  WHERE ManagerID IS NULL

  UNION ALL

  -- Recursive: Find employees managed by those already found
  SELECT e.EmpID, e.Name, e.ManagerID, oc.Level + 1
  FROM Employee e
  INNER JOIN OrgChart oc ON e.ManagerID = oc.EmpID
)
SELECT * FROM OrgChart ORDER BY Level;

```

5. Applications

- **Organizational Charts:** Employee hierarchy traversal.
- **Bill of Materials:** Finding all sub-components of a product.
- **Graph Routing:** Finding connected nodes/paths.

★ Must-Write Points (for 10 marks)

1. Recursive Queries are used for **hierarchical or tree-structured data** (org charts, BoM).
2. Implemented using **Recursive CTE** with the **WITH RECURSIVE** keyword.
3. Three components: **Anchor Member** (base case) + **UNION ALL** + **Recursive Member** (self-referencing).
4. Anchor Member executes once; Recursive Member repeats until it returns an empty set.
5. Final result = UNION ALL of all intermediate result sets.
6. CTE is a temporary named result set — exists only for the duration of one query.
7. Without Recursive CTEs, hierarchical queries required complex procedural code loops.

⚡ Quick Recall

Recursive Query → Hierarchical Data → WITH RECURSIVE → Anchor (base R_0) → UNION ALL → Recursive Member ($R_1, R_2 \dots$) → Terminates when empty → Apps: Org Charts, BoM, Routing